JAVA PROGRAMMING PROJECT

CampusFix

Smart Campus Maintenance Complaint Management System

Submitted in partial fulfilment of the Java Programming course requirements

Team member	Roll number
Member 1 - [Full Name]	[Roll Number 1]
Member 2 - [Full Name]	[Roll Number 2]
Member 3 - [Full Name]	[Roll Number 3]
Member 4 - [Full Name]	[Roll Number 4]

Course: [Course Code / Java Programming]

Semester: [Semester and Academic Year 2026-2027]

Institution: [Institution / Department]

Submission date: 30 November 2026

Demo window: 4-7 December 2026

Project category: Desktop GUI • Object-oriented design • Data persistence

Abstract

CampusFix is a Java desktop application designed to streamline campus maintenance reporting and resolution. Students submit issues such as damaged furniture, electrical faults, and network problems through a structured interface. Administrators monitor tickets, prioritize them by severity, issue category, and waiting time, and assign work to technicians. Technicians track their assigned complaints and update progress until resolution. The project demonstrates object-oriented design with inheritance, interfaces, encapsulation, polymorphism, exception handling, collections, and event-driven Swing components. Complaint records persist in SQLite through JDBC when the driver is available; a local-file repository supports offline demonstrations without external dependencies. A CSV exporter makes issue records portable for analysis. The system is intentionally scoped for a clear five-to-ten-minute academic demonstration while providing a foundation for a future production-ready campus service portal.

Keywords

Java, Swing, OOP, interfaces, JDBC, SQLite, complaint tracking, campus operations, priority scheduling.

Report Contents

1. Introduction and Project Scope
2. Requirements Analysis
3. Use Case and Workflow Diagrams
4. UML Class Diagram
5. Architecture and Database Design
6. Java Implementation and Core Algorithms
7. GUI Screenshots
8. Testing and Evaluation
9. Team Contributions and Demo Plan
10. Conclusion, Future Work, and References

Submission Checklist

- Runnable Java source code, build files, and automated smoke tests
- Minimum-ten-page printable report including requirements and UML diagrams
- Real screenshots of the student and administrator dashboards
- Team details, course, semester, and finalized GitHub repository URL to be added before LMS submission

1. Introduction

1.1 Background and Motivation

Campus facilities often receive maintenance requests through scattered channels such as emails, calls, or informal messages. As a result, reports can be duplicated, critical issues can receive the same attention as routine problems, and students may not know whether a technician has started working on their complaint. CampusFix provides one structured workflow from submission to resolution.

1.2 Problem Statement

The aim is to design an accessible campus complaint-tracking desktop application that distinguishes user roles, preserves data across sessions, and prioritizes maintenance requests transparently. For example, a dangerous electrical outlet should rank above a low-severity cleaning request.

1.3 Objectives

11. Provide separate student, administrator, and technician views.
12. Support validated complaint creation with category, severity, location, and description.
13. Compute priority scores and display the most urgent unresolved work first.
14. Enable controlled assignment and workflow transitions.
15. Persist complaint records and export portable CSV reports.
16. Use Java OOP concepts intentionally and make them explainable during the viva.

1.4 Scope and Assumptions

The system models a single campus with demonstration accounts stored in the application. It is a local desktop project rather than a hosted multi-user service. SQLite provides database persistence on the Maven runtime; a plain-file backup implementation enables a standalone Java demonstration. The password and identities are illustrative for classroom use, not production authentication.

2. Requirement Analysis

2.1 Functional Requirements

ID	Requirement	Responsible role
FR-01	Log in with a role-specific demonstration account	All
FR-02	Submit validated complaint information	Student
FR-03	View only own or assigned complaints	Student / Technician
FR-04	View every complaint and dashboard totals	Administrator
FR-05	Assign an available technician	Administrator
FR-06	Update complaint status through allowed transitions	Technician / Administrator
FR-07	Search and filter the visible complaint list	All
FR-08	Rank unresolved complaints by priority score	Administrator
FR-09	Save complaint data and export CSV	System / All

2.2 Non-functional Requirements

- Usability: large, readable tables and clearly labeled action buttons.
- Correctness: role checks, input validation, and status-transition rules.
- Reliability: complaint persistence after closing and reopening the application.
- Portability: Java 17+ support on Windows, Linux, and macOS with a desktop environment.
- Maintainability: distinct model, service, persistence, GUI, and utility packages.
- Data safety: prepared SQL statements; atomic local-file replacement; no destructive delete feature.

2.3 Required Software and Hardware

A computer with JDK 17 or higher and sufficient memory to run a Swing application is required. Maven and Internet access are needed only for the preferred SQLite JDBC mode on first dependency download. The included run scripts compile a standard-library-only file-storage mode offline.

3. Use Case Diagram

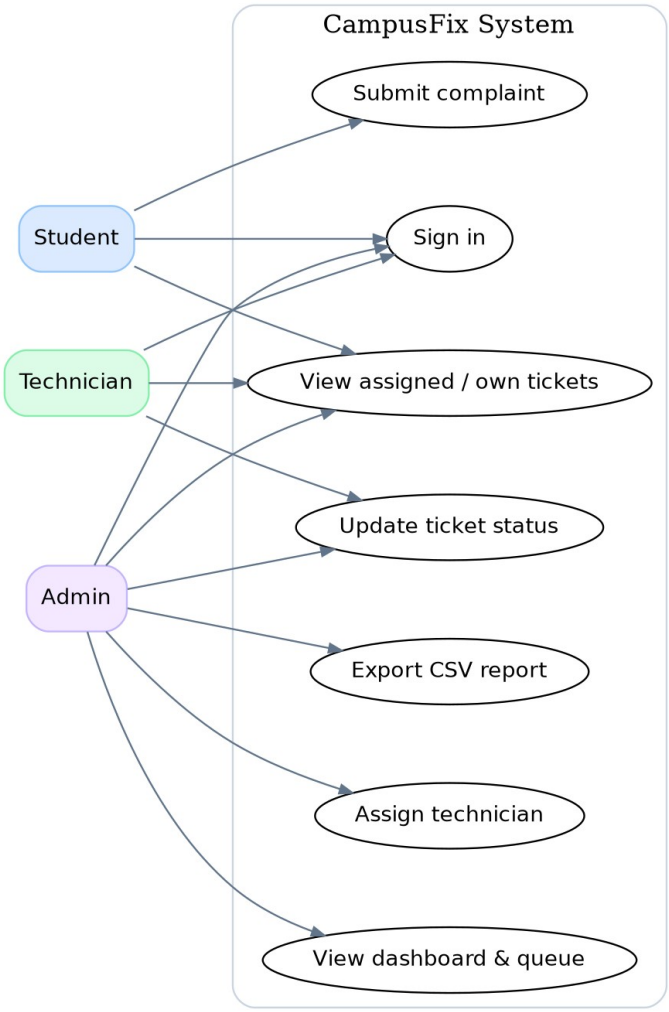


Figure 1. User interactions and access boundaries.

3.1 Main Actors

Student: creates reports and checks their own status. Technician: accesses only assigned work orders and moves them through the repair workflow. Administrator: supervises the entire system, assigns work, exports reports, and views priority-based operational metrics.

3.2 Typical Use Case

A student records a network issue in a computer laboratory. The system validates the title, detailed description, and location, then creates an OPEN complaint. An administrator reviews the prioritized table, selects a technician, and assigns the ticket. The technician moves the request from ASSIGNED to IN_PROGRESS and finally RESOLVED. Both the student and administrator can see the updated result.

4. Workflow and Flowchart

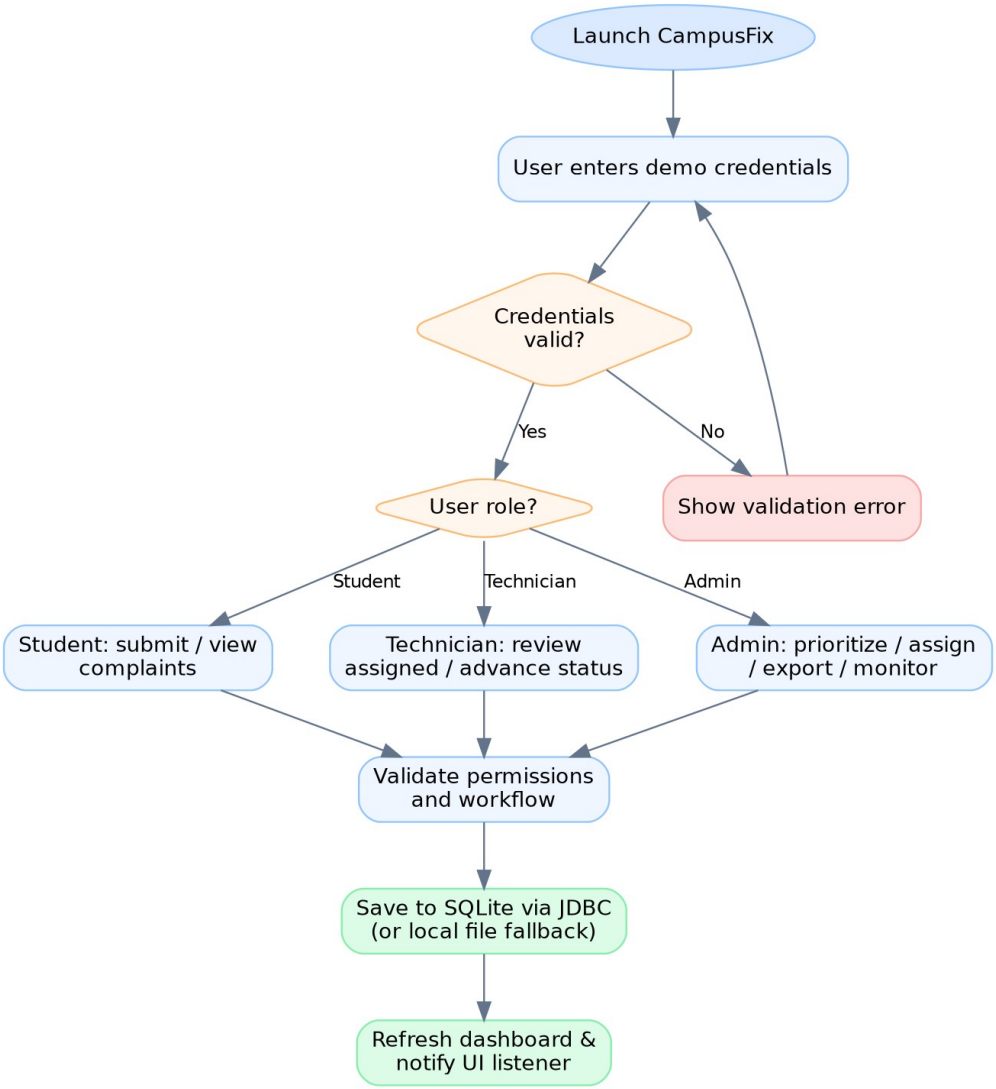


Figure 2. End-to-end complaint lifecycle.

4.1 Status Workflow

The supported sequence is OPEN -> ASSIGNED -> IN_PROGRESS -> RESOLVED. A complaint is assigned by an administrator, after which an assigned technician or administrator can advance it. Unauthorized changes and skipped states result in a ValidationException, which the GUI reports through an alert dialog.

5. UML Class Diagram

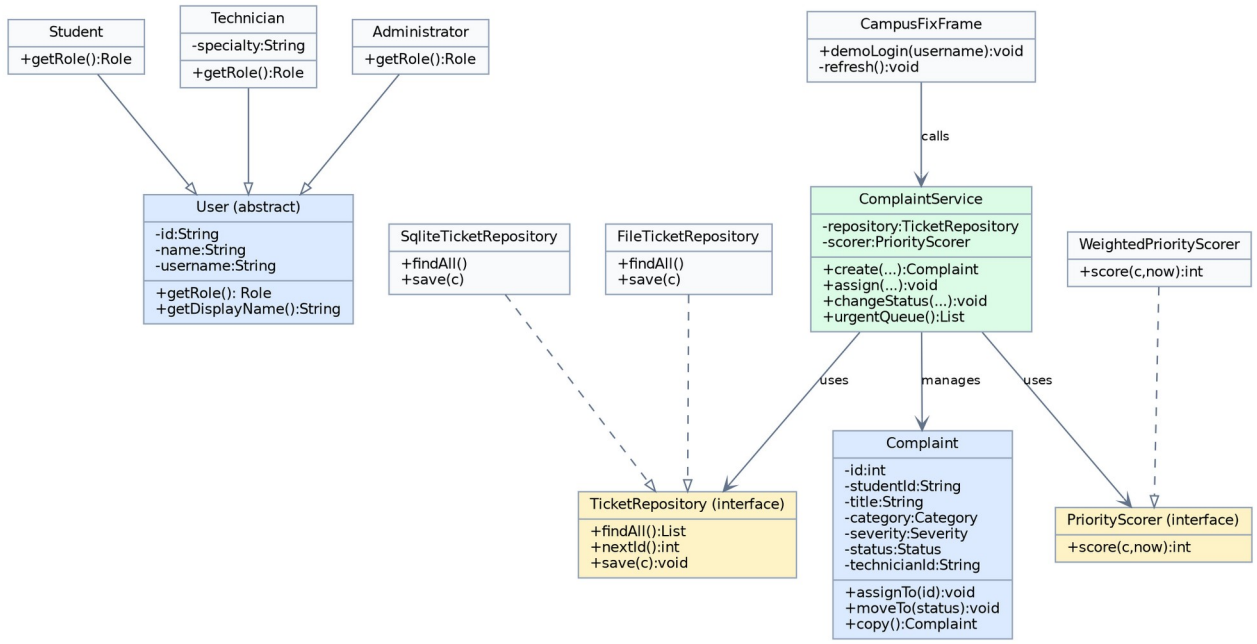


Figure 3. Selected model, repository, service, and GUI class relationships.

5.1 Class Responsibilities

User is an abstract parent; Student, Technician, and Administrator inherit common identity fields and override getRole(). Complaint encapsulates its status, category, severity, location, and assignment details. ComplaintService coordinates permissions and state changes. TicketRepository is an interface implemented by both SqliteTicketRepository and FileTicketRepository. PriorityScorer is a second interface implemented by WeightedPriorityScorer. CampusFixFrame provides the desktop user interface.

5.2 Relationships and OOP Principles

Solid inheritance lines represent an is-a relationship. Dashed realization arrows represent interface implementation. The service depends on abstractions, not a specific database driver, allowing repositories and algorithms to be replaced without rewriting the GUI.

6. System Architecture and Persistence

6.1 Layered Architecture

Layer	Source package	Main responsibility
Presentation	com.campusfix.ui	Swing forms, dashboard, table, dialogs
Service	com.campusfix.service	Business rules, permissions, prioritization
Domain	com.campusfix.model	Users, complaints, enum states
Persistence	com.campusfix.persistence	SQLite / JDBC and file-based storage
Utilities	com.campusfix.util	CSV report export

6.2 SQLite Database Design

The database contains a complaints table. Demo users are seeded in UserDirectory rather than stored in the database; therefore, this version is best considered a single-device demonstration system.

Field	SQLite data type	Meaning
id	INTEGER PRIMARY KEY	Unique complaint ID
student_id	TEXT NOT NULL	Owning student ID
title, description, location	TEXT NOT NULL	Issue details
category, severity	TEXT NOT NULL	Controlled enum names
created_at, updated_at	TEXT NOT NULL	ISO timestamp strings
status	TEXT NOT NULL	Current workflow state
technician_id	TEXT nullable	Assigned technician or null

6.3 Data Durability

SqliteTicketRepository uses DriverManager, PreparedStatement, ResultSet, and an UPSERT to insert or modify complaint records. FileTicketRepository uses buffered I/O, Base64-encoded tab-delimited fields, and replacement of a temporary file to reduce the risk of incomplete writes. CampusFixApp attempts SQLite first and gracefully falls back to file storage if the driver is unavailable.

7. Java Implementation: OOP Concepts

7.1 Encapsulation and Inheritance

User exposes read-only accessors for ID, name, and username. The abstract getRole() method is implemented differently by Student, Technician, and Administrator. Complaint fields are private, and mutation occurs through assignTo() and moveTo(), preventing the GUI from directly changing internal state.

7.2 Interfaces and Polymorphism

TicketRepository has two interchangeable implementations, so ComplaintService depends on an interface. PriorityScorer supports alternate prioritization strategies. TicketListener is a functional interface used by the Swing dashboard to refresh when complaint data changes.

7.3 Exceptions and Validation

ValidationException represents invalid actions such as a title with fewer than five characters, a non-admin attempt to assign technicians, and an illegal status transition. Database failures are wrapped as IllegalStateException. The GUI catches these conditions and shows human-readable messages rather than allowing the application to terminate unexpectedly.

7.4 GUI, Collections, and File Handling

- Swing: JFrame, JTable, JButton, JTextField, JComboBox, dialogs, layout managers, and event listeners.
- Collections: List for user and complaint records; Map for technician workloads; PriorityQueue for prioritized unresolved tickets.
- Streams: filtering complaints by role, sorting by score, and calculating workload summaries.
- File I/O: buffered reading and writing of persistent records, CSV export with correct quoting, and atomic file moves where supported.

7.5 Separation of Concerns

The GUI delegates operations to ComplaintService. The service checks business rules, then invokes the repository. This arrangement supports unit-level testing without requiring the GUI to start.

8. Priority and Assignment Algorithms

8.1 Weighted Priority Score

CampusFix uses a transparent weighted score in the range 0 to 100. Severity contributes LOW=10, MEDIUM=25, HIGH=45, or CRITICAL=65 points. Category contributes ELECTRICAL=15, INTERNET or PLUMBING=10, or other=5 points. Waiting time contributes two additional points for each completed twelve-hour period, up to twenty points. The final value is capped at 100.

Priority = min(100, SeverityPoints + CategoryPoints + min(20, 2 x floor(WaitingHours / 12)))

8.2 Priority Queue

A java.util.PriorityQueue orders unresolved complaints from highest score to lowest score. Equal-scoring complaints are ordered by creation timestamp so earlier requests receive preference. The dashboard table uses the same urgency-oriented ordering.

8.3 Technician Recommendation

The service counts each technician’s tickets that are not RESOLVED. The technician with the smallest current workload is recommended, and administrators can still choose a different technician. This is a simple greedy balancing heuristic; a future edition could also consider skill matches and availability.

8.4 Input and Workflow Validation

17. Reject missing or undersized title, description, and location fields.
18. Only a student may create a ticket; only an administrator may assign it.
19. Only the assigned technician or administrator may advance a ticket.
20. Reject skipping directly from ASSIGNED to RESOLVED or changing a resolved ticket.
21. Store the updated record and refresh the interface after every permitted change.

9. Graphical User Interface - Administrator

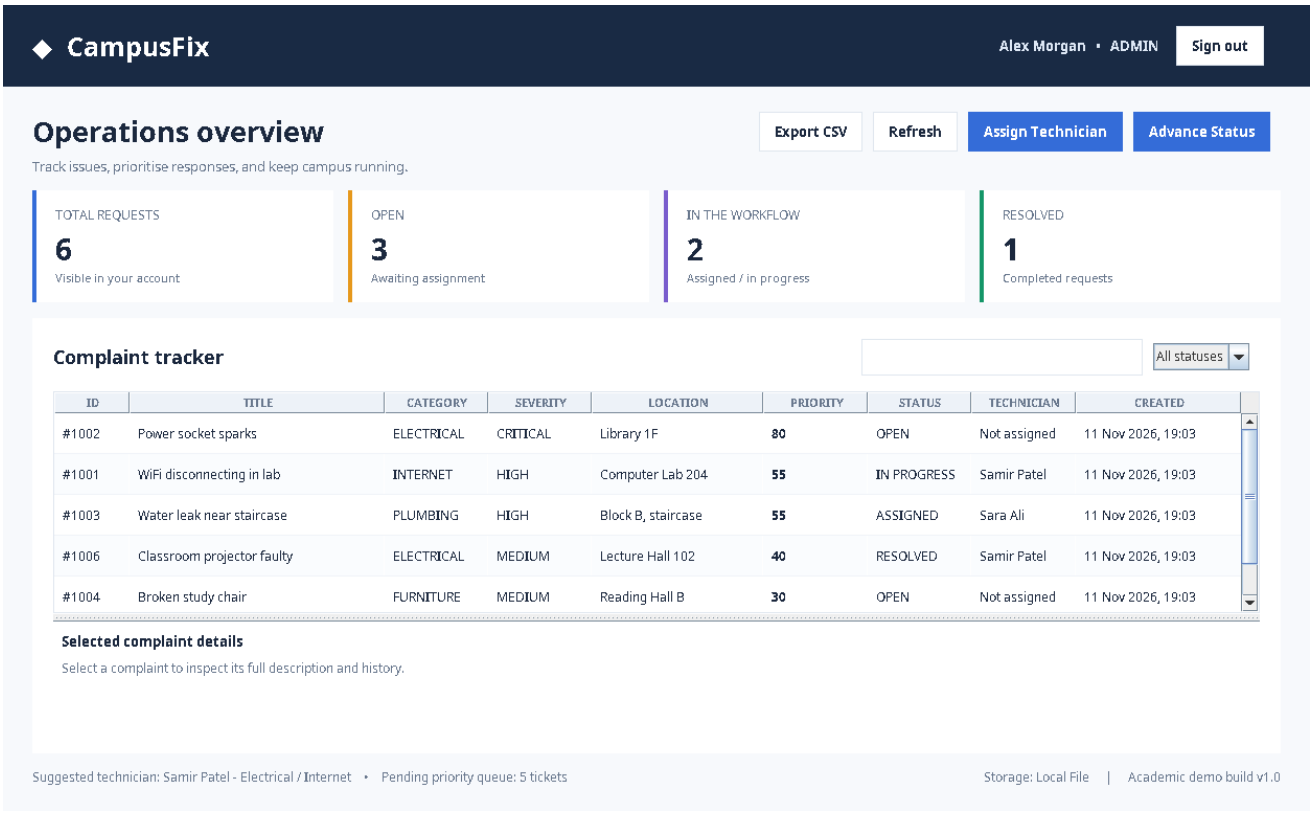


Figure 4. Actual captured CampusFix administrator dashboard running under Java Swing.

Administrator Dashboard Features

The dashboard summarizes total tickets, new OPEN tickets, work in progress, and RESOLVED tickets. The central JTable displays severity, category, priority, assignment, location, and creation time. Administrators can search the table, filter by status, assign technicians, advance the workflow, refresh the view, and export CSV data. The footer identifies the current storage implementation.

Administrator Demo Scenario

Select the high-priority electrical complaint, click Assign Technician, verify the suggested technician, then advance an assigned ticket from ASSIGNED to IN_PROGRESS. The table and summary cards update immediately.

10. Graphical User Interface - Student

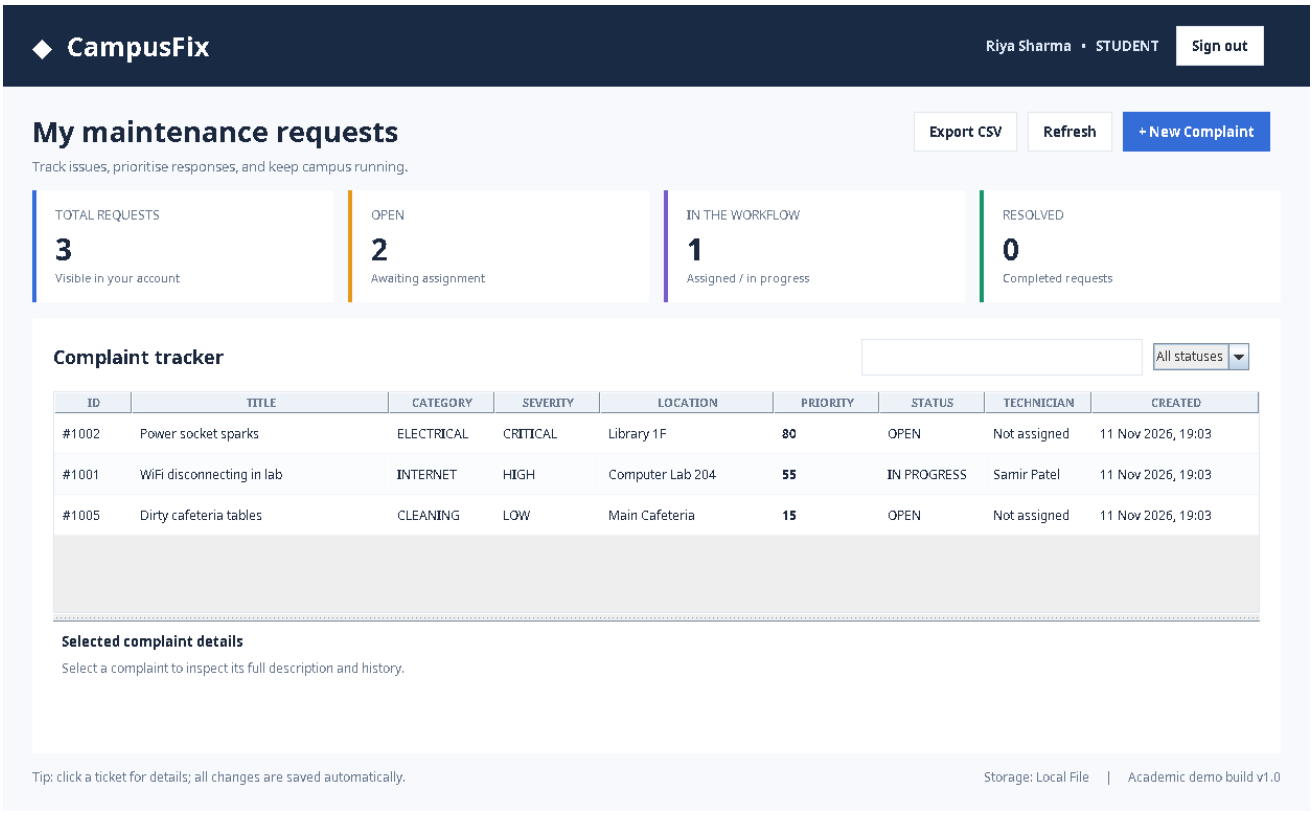


Figure 5. Actual captured CampusFix student dashboard.

Student Dashboard Features

A student views only their own tickets, including those that are OPEN, in progress, or resolved. The New Complaint form requests a short title, detailed description, category, severity level, and location. User input is validated in ComplaintService rather than only in the interface. Students can export their own records but cannot change assignments or ticket statuses.

Student Demo Scenario

Sign in as student with password demo123, click New Complaint, enter a valid electrical, internet, or facilities issue, and submit. Sign in as admin to show that the new ticket becomes visible in the global priority queue.

11. Testing and Evaluation

11.1 Automated Functional Checks

The project includes an executable Java smoke-test suite at src/test/java/com/campusfix/SmokeTest.java. The tests run with the standard Java compiler and do not require an external testing library.

Test ID	Scenario	Observed result
TC-01	Valid / invalid sign-in	PASS
TC-02	Student inheritance and roles	PASS
TC-03	Invalid complaint data rejected	PASS
TC-04	Student / technician visibility	PASS
TC-05	Only admin can assign technician	PASS
TC-06	Illegal status transition rejected	PASS
TC-07	ASSIGNED -> IN_PROGRESS -> RESOLVED	PASS
TC-08	File persistence after reload	PASS
TC-09	Critical ticket outranks low severity	PASS
TC-10	CSV export includes record	PASS

11.2 Execution Evidence

Test command on macOS/Linux: ./test.sh. On Windows: test.bat. The console reports:

PASS: login, inheritance, validation, permissions, assignment, transitions, file persistence, priority scoring, CSV export

11.3 Current Limitations

Testing of SQLite mode requires Maven to download the SQLite JDBC dependency and run the application in that configuration. The current automated suite validates the file repository but does not execute live database integration tests. Authentication is demo-only; live synchronization between several computers, attachments, and notifications are not implemented. These limitations must not be presented as completed features during the demonstration.

12. Team Division and Demonstration

12.1 Work Distribution for Four Members

Member	Primary ownership	Viva concepts
Member 1 - [Name]	Swing GUI: login, dashboard, dialogs, validation feedback	GUI events, JTable, layout managers
Member 2 - [Name]	OOP models, role rules, complaint service, prioritization	Inheritance, polymorphism, interfaces, collections
Member 3 - [Name]	SQLite JDBC, file repository, data export	SQL, JDBC, exceptions, file handling
Member 4 - [Name]	Testing, diagram design, report, integration, demo preparation	Testing strategy, UML, application architecture

All four team members should review each module, contribute to integration, and attend the demonstration together. Final contributor names and roll numbers must be entered on the cover before submission.

12.2 Five-to-Ten-Minute Live Demo

22. 0:00-1:00: Introduce the campus problem and three roles.
23. 1:00-2:30: Log in as a student and submit a new complaint.
24. 2:30-4:30: Log in as admin, inspect the priority queue, assign a technician.
25. 4:30-6:00: Log in as technician and move the assigned complaint forward.
26. 6:00-7:30: Show persistent records and CSV report export.
27. 7:30-9:00: Explain class diagram, interfaces, and exception handling.
28. 9:00-10:00: Answer questions and confirm each member’s contribution.

12.3 Key Dates

Report and code/GitHub link submission: 30 November 2026. Demonstration window: 4-7 December 2026, subject to assigned evaluator schedule. Bring a printed report and a working copy of the project.

13. Conclusion and Future Work

13.1 Conclusion

CampusFix demonstrates a complete academic workflow for reporting and managing campus maintenance issues. The application offers three role-specific dashboards, validated complaint creation, priority-based ordering, administrator assignment, technician status updates, data persistence, and CSV reporting. Its layered architecture separates domain models, business rules, repositories, and GUI components, allowing OOP concepts to be observed directly in the source code and demonstrated through practical actions.

13.2 Future Enhancements

29. Replace demonstration credentials with securely persisted accounts, password hashing, and institutional single sign-on.
30. Introduce an online service or server API for multiple concurrent users and real-time updates.
31. Support attachments, location maps, notification emails, technician availability, and SLA timers.
32. Implement audit history, complaint comments, and richer administrator analytics.
33. Add automated JDBC integration tests, continuous integration, and installer packages.

References

- Oracle. Java Platform, Standard Edition Documentation. https://docs.oracle.com/en/java/
- Oracle. Creating a GUI With Swing. https://docs.oracle.com/javase/tutorial/uiswing/
- Oracle. JDBC Basics. https://docs.oracle.com/javase/tutorial/jdbc/basics/
- Xerial. SQLite JDBC Driver. https://github.com/xerial/sqlite-jdbc
- SQLite Project. SQLite Documentation. https://www.sqlite.org/docs.html
- Apache Maven Project. Maven Documentation. https://maven.apache.org/guides/

Submission Notes

Replace all cover-page placeholders with verified team details. Attach the final GitHub repository link in the LMS submission together with the report. Verify the working project, database mode, report formatting, and printed copy before the demonstration.